IA3 – Software Engineering Project Guidelines

1. Working Software Project

Each team must develop and demonstrate a fully functional software application that addresses a clearly defined problem.

Requirements:

- Identify the problem statement, project objectives and target users.
- Implement the core functionalities of the proposed system.
- Ensure that the application runs successfully and its major modules work together. •

  Include appropriate input validation and error handling.
- Provide sample inputs and data required to demonstrate the application.
- Submit the complete source code along with setup and execution instructions. •

  Demonstrate the working application during the IA3 evaluation.

2. Detailed Software Requirements Specification (SRS)

Each team must prepare a comprehensive SRS document describing the software system to be developed.

The SRS document must include:

1. Introduction: Purpose, scope, objectives and intended audience.
2. Overall Description: Product perspective, user classes, assumptions, constraints and dependencies.
3. Functional Requirements: Detailed descriptions of the features and services provided by the system.
4. Non-Functional Requirements: Performance, security, usability, reliability and other relevant quality requirements.
5. External Interface Requirements: User interfaces, hardware interfaces, software interfaces and communication interfaces, where applicable.
6. Data Requirements: Data inputs, outputs, storage requirements and database specifications, where applicable.
7. Use Cases: Descriptions of user interactions and system behaviour.
8. Requirement Traceability: Unique identifiers for requirements and their relationship to design and testing.
9. Document References: Relevant references, terminology and supporting information.

The document must be clearly structured, detailed, consistent and specific to the team's project. Requirements should be unambiguous and testable.

3. UML Diagrams

Each team must prepare five UML diagrams representing the design and behaviour of the proposed

software system.

1. Use Case Diagram

Identify the system's actors, use cases, system boundary and relationships between users and system functionalities.

2. Class Diagram

Represent the system's classes, attributes, operations, associations, inheritance and other relevant relationships.

3. Sequence Diagram

Illustrate the order of interactions and messages exchanged between actors, objects and system components for an important scenario.

4. Activity Diagram

Represent the workflow of a major process, including actions, decision points, branches and start/end nodes.

5. State Machine Diagram

Illustrate the significant states of a system object and the events or conditions that trigger transitions between states.

UML documentation requirements:

- Use standard UML notation.
- Ensure that each diagram is readable, properly labelled and relevant to the project. •

  Maintain consistency between the UML diagrams, SRS and implemented application. •

  Include a title and a brief explanation of each diagram.
- Use a component or deployment diagram instead of a state machine diagram only when approved and justified by the project requirements.

4. Detailed Software Test Cases

Each team must prepare a test case document that describes how the system's functionality will be verified and validated.

The test case document must include:

- Unique test case ID.
- Requirement ID associated with the test case.
- Test objective or description.
- Preconditions.
- Step-by-step test procedure.
- Input data or test data.
- Expected result.

- Actual result obtained during execution.
- Test status (Pass/Fail).
- Remarks, defect details or supporting evidence, where applicable.

The test suite must cover:

1. Positive testing: Valid inputs and expected successful operations.
2. Negative testing: Invalid inputs, incorrect credentials and unexpected conditions. 3.

Boundary testing: Minimum, maximum and edge-case values, wherever applicable. 4.

Functional testing: Verification of the major features and workflows.

5. Integration testing: Verification that interacting modules work correctly together.
6. Validation and error handling: Verification of error messages, input restrictions and recovery behaviour.

Test cases must be executed against the working application. The document should record actual outcomes and identify failed tests or defects rather than listing only expected results.